

SafeContract: A Practitioner’s Toolkit for VLA Action-Space Monitoring

Krishnam Gupta
Independent Research
San Francisco, CA
kayjee1994@gmail.com

Abstract—Vision-Language-Action models ship with zero action-space safety infrastructure. OpenVLA sends raw outputs to motors with no bounds checking; pi0 streams via WebSocket with no validation; LeRobot’s safety processor is imperative and easy to omit. We present SafeContract, a training-free, black-box toolkit that adds calibrated action monitoring to any VLA in four lines of Python. Across three architectures (VQ-BeT, Diffusion Policy, ACT) and two environments (PushT 2D, ALOHA 14-DOF bimanual), we find that different architectures need different monitors: direction reversal rate is a universal failure predictor (AUROC 0.93, 0.79, 0.91), jerk predicts failure only for discrete-token models (0.88 vs. 0.43), and velocity violations alone are non-predictive (0.41–0.69). Split-conformal calibration from demonstrations eliminates hand-tuned thresholds (97.9% coverage), and CUSUM shift detection catches degradation within 7 steps. SafeContract enforces and monitors at $13\mu\text{s}$ per step - less than 0.001% of typical VLA inference time - with no model access, no retraining, and no task degradation ($p \geq 0.68$ across all experiments).¹

Index Terms—VLA deployment, action-space monitoring, safety toolkit, conformal prediction, failure prediction

I. Introduction: The Deployment Gap

VLA models [1], [2] predict robot actions end-to-end from vision and language. The field has invested heavily in making these models more capable, but virtually nothing in making their outputs safe at deployment time. Consider the current state of practice:

- OpenVLA’s deployment script sends raw action outputs directly to motors with zero bounds checking [1].
- pi0 streams actions via WebSocket with no validation layer [4].
- LeRobot’s [3] EEBoundsAndSafety processor is imperative, non-composable, and trivially omitted.
- SmolVLA produces actions up to $3.7\times$ outside $[-1, 1]$ bounds on LIBERO [5].

The implicit assumption is: if the model is good enough, the actions will be safe enough. We show this assumption is wrong, and that it is wrong in architecture-specific ways that a one-size-fits-all monitor cannot catch.

This paper answers one question: “I am deploying a VLA tomorrow. What should I do?” We provide (1) a four-step deployment workflow, (2) architecture-matched monitor recommendations backed by controlled experiments, and (3) an open-source toolkit that implements the full pipeline in four lines of Python.

Our contributions:

- 1) Architecture-matched monitoring recommendations. Same-dataset experiments ($n=200$ episodes, same seeds) show VQ-BeT produces $2.4\times$ more velocity violations than Diffusion Policy, and the best monitor differs by architecture family (Table I).
- 2) A universal failure predictor. Direction reversal rate achieves AUROC = 0.93 (VQ-BeT), 0.79 (Diffusion), and 0.91 (ACT/ALOHA 14-DOF) - the only monitor predictive across all three architectures.
- 3) Data-driven calibration that replaces hand-tuning. Split-conformal bounds [15] achieve 97.9% coverage; CUSUM [16] detects degradation within 7 steps. No hand-tuned thresholds, no model internals.

II. Deployment Workflow

SafeContract turns VLA deployment monitoring into four steps (Fig. 1).

Step 1: Calibrate from demonstrations. `SafetyGuard.from_demos(demo_actions)` computes split-conformal bounds ($\alpha=0.05$) and per-joint velocity limits (99th percentile of consecutive deltas) from your demonstration data. No manual threshold tuning. At $\alpha=0.05$, conformal bounds are 25% tighter than the 4σ heuristic while providing 97.9% holdout coverage (99.1% minimum per dimension).

Step 2: Select monitors for your architecture. Consult Table II. If your VLA uses discrete tokens (VQ-BeT, OpenVLA), enable jerk and reversal monitors. If it uses continuous denoising (Diffusion, flow matching), enable stall detection and reversal monitors. If you are unsure, enable all monitors - the overhead is negligible.

Step 3: Wrap your predict function.

```
guard = SafetyGuard.from_demos(demo_actions)
```

```
@guard.wrap # bounds + velocity + stall + jerk
def predict(obs):
    return model(obs)
```

```
# Every call is now monitored and enforced.
action = predict(obs)
status = guard.status() # p-value, CUSUM, flags
```

The decorator clips to calibrated bounds, clamps velocity, and logs all violations with timestep, dimension, and

¹Code and results: <https://github.com/krishnam94/vla-edge>

SmolVLA outputs exceed safe bounds on LIBERO

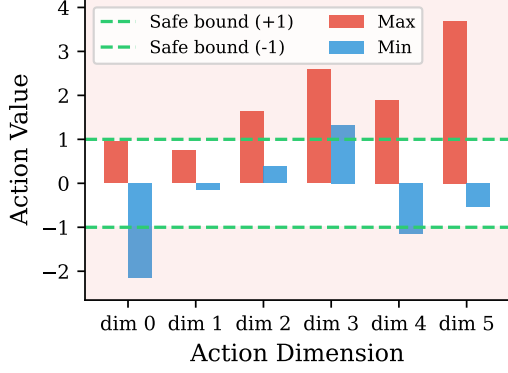


Fig. 1: SmolVLA action ranges on 100 LIBERO observations. Most dimensions exceed $[-1, 1]$ bounds (green dashes), confirming that VLAs produce unsafe actions that monitoring would catch.

magnitude. Overhead: $13\mu\text{s}$ per call ($<0.001\%$ of VLA inference).

Step 4: Monitor CUSUM during rollout. The conformal p-value stream feeds a CUSUM detector that accumulates evidence of distribution shift. When CUSUM exceeds threshold h , the system signals degradation. Response options: reduce $v_{\max}$, switch to a fallback controller, or request human intervention. On ALOHA demonstrations, in-distribution actions produce zero alarms; adding $\sigma=0.1$ noise triggers detection within 7 steps ($h=3$).

III. Method

A. Safety Contracts and Enforcement

A safety contract $C = (\mathbf{l}, \mathbf{u}, v_{\max})$ specifies per-joint bounds and a velocity limit: $l_i \leq a_i \leq u_i$ for each joint i , and $\|\mathbf{a}_t - \mathbf{a}_{t-1}\|_{\infty} \leq v_{\max}$. Enforcement proceeds in three phases: (1) clip to per-joint bounds, (2) clamp velocity, (3) re-clip to bounds. The final re-clip is critical: velocity clamping can push actions outside original bounds. For hyperrectangular contracts C_1, C_2 , sequential clipping equals intersection clipping, so stacking decorators is order-independent.

B. Conformal Calibration

Hand-tuned thresholds are unreliable: $v_{\max}=0.05$ clips a significant fraction of expert actions on ALOHA. Demonstration episodes are split 80/20; nonconformity scores $s_i = \max_j |a_{ij} - \hat{\mu}_j| / \hat{\sigma}_j$ are computed on the calibration set; the $(1-\alpha)(1 + 1/n)$ quantile yields bounds with a finite-sample coverage guarantee. Velocity limits are set per-joint at the 99th percentile of consecutive action deltas.

Each action also receives a conformal p-value: $p(s) = |\{i : s_i \geq s\}| / (n+1)$. Under exchangeability, $P(p \leq \alpha) \leq \alpha$ - a proper statistical test, not a heuristic [15]. The p-value stream feeds CUSUM [16]: $S_t = \max(0, S_{t-1} + \mathbf{1}[p_t <$

$\alpha] - \alpha)$, which is Lorden-optimal for detecting persistent mean shifts, with a formal false-alarm bound that EWMA cannot provide.

C. Architecture-Specific Monitors

Bounds and velocity alone miss architecture-specific failure modes. SafeContract includes four targeted monitors: Direction reversal rate. For each joint j , a reversal occurs when $\text{sign}(\Delta a_{t,j}) \neq \text{sign}(\Delta a_{t-1,j})$. The episode rate averages over all (timestep, joint) pairs. High reversal rates indicate indecisive, oscillatory behavior. This is the single most predictive monitor across all architectures we tested. Jerk monitoring. The third derivative of the action trajectory detects denoising artifacts in diffusion policies and chunk boundary discontinuities in action-chunked models [21]. Strongly predictive for discrete-token architectures; less so for continuous denoisers.

Stall detection. Flags when $\|\mathbf{a}_t - \mathbf{a}_{t-1}\|_2 < \tau$ for w consecutive steps. Catches policy freeze - the primary failure mode of diffusion policies [17] - which is invisible to bounds/velocity monitoring.

Gripper oscillation. Rapid open/close cycling ($> k$ flips in w steps) signals grasp indecision in bimanual manipulation. Individual transitions pass velocity checks, but the pattern is pathological.

IV. Experiments

Platform: RunPod GPU cluster, Python 3.12. Full monitoring overhead: $<0.001\%$ of typical VLA inference time ($\sim 13\mu\text{s}$ per call).

A. VLAs Produce Unsafe Actions

SmolVLA on 100 LIBERO observations: after correct MEAN_STD unnormalization, bounds violations drop from 86% to 0% - every bounds violation was a normalization bug, detectable by monitoring. The robust finding is velocity: at $v_{\max}=0.1$ rad/step, 98% of SmolVLA steps violate versus 16% of expert steps - a $6\times$ gap that persists regardless of normalization. Bounds monitoring catches integration bugs; velocity monitoring catches physical safety concerns.

B. Cross-Architecture Comparison

We evaluate three architectures on their respective tasks, using conformal calibration from demonstrations (Step 1) and the same per-architecture evaluation protocol.

Diffusion Policy [23] (262M, PushT, $n=200$, seeds 0–199). Without SafeContract: 58% success. With: 57% ($p = 0.92$). 772 violations caught, zero task degradation.

VQ-BeT [24] (37.5M, PushT, $n=200$, same seeds). Without: 56.5%. With: 54.5% ($p = 0.76$). 1,847 violations - $2.4\times$ more than Diffusion on the same task.

ACT [25] (51.6M, ALOHA 14-DOF bimanual, $n=50$). Without: 62%. With: 56% ($p = 0.68$). 4,758 violations caught.

Across all three, SafeContract enforces without statistically significant task degradation.

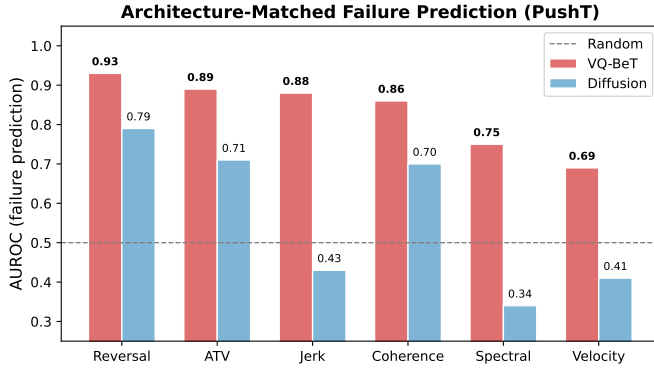


Fig. 2: Monitor effectiveness varies by architecture. Reversal rate is the universal predictor. Jerk works for discrete-token models (VQ-BeT) but not continuous denoisers (Diffusion). No single monitor is best everywhere.

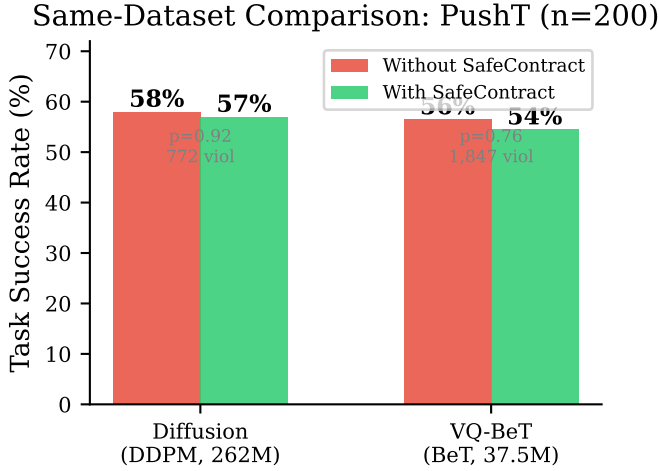


Fig. 3: Same-dataset comparison on PushT ($n=200$). SafeContract catches 772 (Diffusion) and 1,847 (VQ-BeT) violations with no task degradation ($p=0.92$, $p=0.76$).

C. Which Monitors Work for Which Architecture?

Table I presents failure prediction AUROCs. The key findings for practitioners:

Always enable reversal rate monitoring. It predicts failure with AUROC 0.93 (VQ-BeT), 0.79 (Diffusion), and 0.91 (ACT/ALOHA). It is the only monitor that is strongly predictive ($p < 0.001$) across all three architectures and both action dimensionalities (2D and 14-DOF).

Enable jerk monitoring for discrete-token VLAs. Jerk AUROC follows a discrete-to-continuous gradient: 0.88 (VQ-BeT), 0.69 (ACT), 0.43 (Diffusion). For autoregressive token-based VLAs (OpenVLA, VQ-BeT), jerk captures quantization artifacts at token boundaries. For smooth denoisers, jerk is non-predictive.

Do not rely on velocity violations alone. Velocity AUROC ranges from 0.41 to 0.69 - not useful as a standalone failure predictor for any architecture.

V. Practical Recommendations

Table II summarizes monitor selection by VLA architecture family. The recommendations extend beyond the three architectures we tested: they are grounded in the structural properties (discrete vs. continuous action generation, chunked vs. per-step) that determine failure modes.

Deployment integration. SafeContract operates at the action output layer, between the VLA’s `predict()` and the robot’s motor commands. It is agnostic to the middleware above (ROS 2, LeRobot, OpenPI, direct motor control) and below (Jetson, GPU server, cloud). For ROS 2, wrap the action callback. For LeRobot, decorate the policy’s `select_action()`. For direct motor control, insert between inference and the motor driver.

Where SafeContract fits in a full monitoring stack. A production VLA deployment needs monitoring at multiple levels [22]: workspace constraints (CBFs), task progress (semantic monitors), perception consistency, and more. SafeContract is Layer 0 - the innermost, fastest check on raw action outputs. It composes with CBF-based workspace constraints [7], task-level monitors [14], and perception-level failure detectors [18] as part of defense-in-depth. Its value is that it requires nothing except action outputs: no dynamics model, no model internals, no auxiliary training.

VI. Related Work

Training-time safety. SafeVLA [6] and RobustVLA [9] improve safety via retraining or fine-tuning. Complementary to runtime monitoring - a policy can be both trained safely and monitored at deployment.

Inference-time safety. AEGIS [7] solves a CBF-QP per step (requires dynamics models); SafeDiffuser [11] and CoDiG [12] embed barriers in diffusion (architecture-specific); SafeDec [8] constrains decoding; Safety Chip [10] uses LTL monitors; ATACOM [13] projects onto constraint manifolds. All require either dynamics models, architecture access, or retraining. SafeContract requires none of these.

Failure detection. SAFE [17] and FIPER [18] train auxiliary models on VLA features for failure detection; Sentinel [14] monitors temporal consistency. These detect failures but do not prevent them. SafeContract both detects (via monitors) and prevents (via enforcement).

Conformal prediction for robotics. Recent work validates conformal prediction for robot safety [17], [19]. Kim et al. [22] argue that modular guardrails are necessary for foundation-model robots. SafeContract provides a concrete, calibrated, open-source implementation.

VII. Discussion

The central result is a practical recipe, not just an empirical finding. VLA architectures fail in architecture-specific ways (Table I), and the monitors that predict failure are architecture-specific (Table II). A practitioner

TABLE I: Cross-architecture results with failure prediction AUROCs and practitioner recommendations. PushT: $n=200$ per condition (same seeds/contract), AUROCs on $n=100$. ALOHA: $n=50$ per condition, AUROCs on $n=50$.

	Diffusion (PushT)	VQ-BeT (PushT)	ACT (ALOHA 14-DOF)
Family	Continuous (DDPM)	Discrete (VQ-VAE + Transformer)	Continuous (chunked)
Params	262M	37.5M	51.6M
Success (no / with contract)	58% / 57%	56.5% / 54.5%	62% / 56%
Fisher’s p	0.92	0.76	0.68
Violations caught	772	1,847	4,758
AUROC: reversal rate	0.79	0.93	0.91
AUROC: jerk	0.43	0.88	0.69
AUROC: momentum coherence	0.70	0.86	0.71
AUROC: spectral energy	0.34	0.75	0.74
AUROC: velocity	0.41	0.69	0.52
Recommended monitors	reversal, stall, coherence	reversal, jerk, spectral	reversal, jerk, gripper osc.
Primary failure mode	policy freeze (stall)	token boundary jitter	chunk boundary + grasp indecis.
Watch for	micro-movements, no progress	high-frequency oscillation	gripper cycling, arm drift

TABLE II: Monitor recipes by VLA family. Start with your architecture row. Enable the listed monitors. Watch for the listed failure mode.

VLA Family	Enable These Monitors	Failure Mode to Watch
Discrete token (VQ-BeT, OpenVLA)	reversal rate, jerk, spectral energy	High-freq oscillation at token boundaries
Diffusion / flow (Diffusion, pi0)	reversal rate, stall, momentum coherence	Policy freeze: valid-looking micro-movements
Action-chunked (ACT, GR00T)	reversal rate, jerk, gripper oscillation	Chunk boundary discontinuity, grasp cycling
Hybrid AR+diff (pi0.5, CogACT)	all monitors	Semantically correct trajectory toward wrong target

deploying a discrete-token VLA should watch jerk; a practitioner deploying a diffusion VLA should watch for stalls. Both should always monitor reversal rate.

Regulatory motivation. The EU AI Act (Article 9) mandates continuous monitoring for high-risk AI systems by 2027. Robots operating near humans qualify. SafeContract provides auditable telemetry - per-step violation logs, conformal p-values, CUSUM alarm history - that supports compliance without requiring model transparency.

What SafeContract cannot catch. Hybrid architectures (autoregressive reasoning plus diffusion actions) can produce physically valid trajectories toward the wrong target. No action-space monitor can detect this. Semantic correctness requires perception-level [18] and task-level [14] monitoring above SafeContract in the stack.

Limitations. (1) All experiments are in simulation; real-robot patterns may differ. (2) Three of eleven known VLA architecture families tested; autoregressive (OpenVLA)

and flow matching (pi0) are untested but predicted by the discrete/continuous gradient. (3) Conformal coverage (97.9%) holds under exchangeability with calibration data; during rollout, coverage degrades under distribution shift. CUSUM detects when this occurs; adaptive conformal inference [20] would maintain coverage online.

Future work. Extending to autoregressive and flow-matching architectures, real-robot validation on physical hardware, adaptive conformal inference [20] for online recalibration, and integration with ROS 2 safety lifecycle management.

References

- [1] Kim, M.J. et al., “OpenVLA: An Open-Source Vision-Language-Action Model,” CoRL 2024, arXiv:2406.09246.
- [2] Shukor, M. et al., “SmolVLA: A Vision-Language-Action Model for Affordable and Efficient Robotics,” arXiv:2506.01844, 2025.
- [3] Cadene, R. et al., “LeRobot: State-of-the-art Machine Learning for Real-World Robotics in PyTorch,” 2024, <https://github.com/huggingface/lerobot>.
- [4] Black, K. et al., “ π_0 : A Vision-Language-Action Flow Model for General Robot Control,” arXiv:2410.24164, 2024.
- [5] Liu, B. et al., “LIBERO: Benchmarking Knowledge Transfer for Lifelong Robot Learning,” NeurIPS 2023.
- [6] Zhang, B. et al., “SafeVLA: Towards Safety Alignment of Vision-Language-Action Model via Constrained Learning,” NeurIPS 2025, arXiv:2503.03480.
- [7] Hu, S. et al., “VLSA: Vision-Language-Action Models with Plug-and-Play Safety Constraint Layer,” arXiv:2512.11891, 2025.
- [8] Kapoor, P. et al., “Constrained Decoding for Robotics Foundation Models,” arXiv:2509.01728, 2025.
- [9] Guo, J. et al., “On Robustness of VLA Models against Multi-Modal Perturbations,” ICLR 2026.
- [10] Yang, Z. et al., “Plug in the Safety Chip: Enforcing Constraints for LLM-driven Robot Agents,” ICRA 2024.
- [11] Xiao, W. et al., “SafeDiffuser: Safe Planning with Diffusion Probabilistic Models,” ICLR 2025.
- [12] Ma, H. et al., “CoDiG: Constraint-Aware Diffusion Guidance for Robotics,” CoRL 2025.
- [13] Tolle, M. et al., “Towards Safe Robot Foundation Models Using Inductive Biases,” arXiv:2505.10219, 2025.
- [14] Agia, C. et al., “Unpacking Failure Modes of Generative Policies: Runtime Monitoring of Consistency and Progress,” CoRL 2024, arXiv:2410.04640.
- [15] Vovk, V. et al., “Algorithmic Learning in a Random World,” Springer, 2005.

- [16] Page, E.S., “Continuous Inspection Schemes,” *Biometrika* 41(1/2):100–115, 1954.
- [17] Gu, Q. et al., “SAFE: Multitask Failure Detection for Vision-Language-Action Models,” *NeurIPS 2025*, arXiv:2506.09937.
- [18] Romer, R. et al., “Failure Prediction at Runtime for Generative Robot Policies,” *NeurIPS 2025*, arXiv:2510.09459.
- [19] Feldman, A.O. et al., “Conformal Safety Monitoring for Flight Testing,” arXiv:2511.20811, 2025.
- [20] Gibbs, I. and Candes, E., “Adaptive Conformal Inference Under Distribution Shift,” *NeurIPS 2021*, arXiv:2106.00170.
- [21] Park, M. et al., “ACG: Action Coherence Guidance for Flow-based VLA Models,” arXiv:2510.22201, 2025.
- [22] Kim, J. et al., “Modular Safety Guardrails Are Necessary for Foundation-Model-Enabled Robots in the Real World,” arXiv:2602.04056, 2026.
- [23] Chi, C. et al., “Diffusion Policy: Visuomotor Policy Learning via Action Diffusion,” *RSS 2023*, arXiv:2303.04137.
- [24] Lee, S. et al., “Behavior Generation with Latent Actions,” *ICML 2024*, arXiv:2403.03181.
- [25] Zhao, T.Z. et al., “Learning Fine-Grained Bimanual Manipulation with Low-Cost Hardware,” *RSS 2023*, arXiv:2304.13705.